

kci-dev:

What Changed, What Works, and What Kernel Developers Still Need

Arisu Tachibana

kci-dev creator and project lead

Kernel Testing & Dependability MC / LPC 2026

Our work since LPC 2025

- **Reliable automation:** fixed hangs in job watching and bisection.
- **Clearer results:** corrected false success reports, validation and JSON output.
- **Python API:** lets existing tools use kci-dev without parsing terminal output.
- **Broader workflows:** added external build reporting and patch submission.

A maintainer's question

What changed for my kernel?

- Which revision and configuration did we test?
- Which failures appeared with my change?
- Do we have enough evidence to act?

kci-dev brings KernelCI data into scripts and review workflows.

What changed since LPC 2025

Available by v0.1.11	What it enables
Python client	Import kci-dev into other tools
KCIDB submission + storage	Report external builds and upload artifacts
Patchset submission	Test local patches or allowed patch URLs
Validation + workflow fixes	Check consistency and improve automation

Current main: structured comparison reports and CI gates

Source snapshot: 6 October 2026, `e4c0087`. MCP remains experimental.

A reusable Python interface

```
from kcidb import KernelCIClient

client = KernelCIClient()
summary = client.get_summary(
    origin="maestro", giturl=GIT_URL,
    branch=BRANCH, commit=COMMIT,
)
```

Dashboard queries return Python objects.

`submit_build(...)` reports an external build to KCIDB.

Example plugins using the Python API

Application	Workflow it adds	Shared client methods
<code>kci-patchwork</code>	Test a Patchwork series on a known base	<code>trigger_patchset</code> , <code>get_node</code> , <code>get_nodes</code>
<code>kci_release_review</code>	Compare tested revisions and publish reports	<code>compare_results</code> , <code>get_build</code> , <code>get_test</code> , <code>get_log</code>

Standalone Python applications importing `KernelCIClient` .

Patches on a known base

```
kci-dev patchset --nodeid "$CHECKOUT_NODE" \  
  --patch 0001-fix.patch \  
  --patch 0002-test.patch \  
  --job-filter "$JOB_FILTER" \  
  --watch --test "$TEST_PATH"
```

Requires an existing checkout and a configured pipeline/token.

`--patchurl` accepts allowed URLs, including Patchwork mbox URLs.

Choosing the base and the jobs

- The checkout identifies the unpatched source revision and tarball.
- The series must apply to that revision.
- Job filters select builds and tests available on that instance.
- The configuration must exercise the code the series changes.

A passing build provides evidence for the configuration that ran.

kci-patchwork: a series workflow

1. Resolve the Patchwork URL to a complete series.
2. Validate patch order and freeze the exact diffs.
3. Record the base, selected jobs and submission identifiers.
4. Collect the patched tree and compare completed results.

`manifest.json` records the inputs. HTML and JSON reports retain the evidence.

CIP series submission

```
kci-patchwork \  
  "https://patchwork.kernel.org/project/cip-dev/list/?series=1178390" \  
  --config ~/.config/kci-dev/kci-dev.toml \  
  --instance staging \  
  --checkout 6ab272040ed308c6c2d206c3 \  
  --job kbuild-gcc-14-arm64-510-cip \  
  --submit
```

Captured command: one series, an explicit base and a selected CIP build.

Captured Patchwork run

```
{
  "status": "RUNNING", "exit_code": 3,
  "terminal": false,
  "counts": {"total": 2, "descendants": 1,
    "states": {"closing": 1, "running": 1},
    "results": {"pass": 1, "unset": 1}},
  "comparison": {"performed": false, "complete": false}
}
```

Submission and result collection are active.

The completed build comparison is still pending in this capture.

Continuing the same Patchwork run

```
RUN=runs/patchwork.kernel.org/series-1178390  
kci-patchwork status --run "$RUN"  
kci-patchwork watch --run "$RUN"
```

- `report.html`: readable results and baseline comparisons
- `report.json`: nodes, classifications and collection errors
- The saved run retains the submitted node and tree IDs.

Patchset identity in the Python API

```
response = client.trigger_patchset(  
    nodeid=base_id, patches=ordered_diffs,  
    job_filter=[job],  
)  
patchset = response["node"]  
root = client.get_node(patchset["id"])
```

Patched and unpatched trees can share a Git commit hash.
Collection also uses the patchset hash, tree ID and parent chain.

Comparing two revisions

```
kci-dev results compare \  
  --giturl "$GIT_URL" --branch "$BRANCH" \  
  --format json "$BASE" "$HEAD"
```

Current main: explicit full commit hashes and structured output

Matches origin, platform, arch, compiler, config and path.

Keeps repeated results and their IDs.

What the report tells us

Category	Interpretation
<code>regression</code>	PASS becomes FAIL/ERROR, or history signal
<code>fixed</code>	Recovery to PASS
<code>persistent_fail</code>	Failure on both revisions
<code>unstable</code>	History signal or another status change
<code>new</code>	Result appears only in HEAD
<code>missing</code>	Result appears only in BASE

A regression label needs investigation.

`FAIL` and `ERROR` both count as failures in this classifier.

kci_release_review: tested revisions

- Select a baseline and candidate by full commit hash.
- Reuse kci-dev's comparison and result classifications.
- Collect selected result details and bounded log excerpts.
- Publish portable HTML with the complete JSON report.

The application queries existing results through the Python API.

A report built on the shared client

```
from kci_release_review.client import ReviewClient
from kci_release_review.report import collect_report

client = ReviewClient(max_issue_lookups=20)
report = collect_report(
    client, selection, max_evidence=3, log_bytes=8192,
)
```

`ReviewClient` extends `KernelCIClient`.

`collect_report()` calls `compare_results()` and keeps its evidence.

Published release-review results

linux-6.6.y: v6.6.157 to v6.6.158

Regression 1	Fixed 1	Unstable 0
Persistent Fail 870	New 2573	Missing 1047

Status: **REVIEW_REQUIRED**. Report collected on 5 October 2026.

[Open the published reports](#)

One candidate to investigate

kernelci_watchdog_reset.wdt-reset.wdt-get-timeout

Context	Captured result
Platform	mt8195-cherry-tomato-r2
Architecture / compiler	ARM64 / GCC 14
Baseline	PASS
Candidate	FAIL
Known issues	Not checked in this report

The report links both result IDs for follow-up.

A CIP result with no review signals

v6.12.108-cip31 to v6.12.111-cip32

Observed results	Baseline	Candidate
Builds	6 PASS	6 PASS
Boots	0	15 PASS
Tests	0	89 PASS, 11 SKIP

115 new entries, with no observed failures or missing results.
Required coverage remains unassessed.

Incomplete evidence stays visible

linux-6.12.y: v6.12.110 to v6.12.111

- The report retains 1 regression candidate and 36 recoveries.
- Returned tree history belongs to a different candidate.
- The issue budget leaves 370 result IDs unchecked.

Status: **EVIDENCE_INCOMPLETE**, exit code **2**.

The exact-commit results remain available for review.

Gate policy and coverage

```
kci-dev results gate \  
  --giturl "$GIT_URL" --branch "$BRANCH" \  
  --base "$BASE" --head "$HEAD" \  
  --fail-on regression,missing --format json
```

Exit	Meaning in current main
0	No selected category triggered the policy
1	A selected category triggered the policy
2	Incomplete comparison or command error

Expected coverage still needs an explicit test plan.

What the plugins expose

- Release review checks that history belongs to the selected head.
- Patch review keeps patchset and baseline node identities separate.
- Reports preserve partial results, skipped work and collection errors.
- Shared API fixes can benefit every application using the client.

Consistent configuration and result identity remain core requirements.

A contract for the next plugin

Shared kci-dev interface	Application responsibility
Service requests and typed failures	Input selection and recovery
Explicit endpoint configuration	Credential and instance choices
Result identities and classifications	Required coverage and review policy
Structured Python data	Reports and integration with other tools

Pin the library revision and retain the inputs behind each report.

MCP as an experimental interface

```
pip install 'kci-dev[mcp]'  
kci-dev mcp
```

- Query results, hardware and known issues
- Inspect Maestro nodes with configured API access
- Trigger or retry jobs with pipeline access and a token

Tool names and response formats can change.

Proposed priorities

Workstream	Concrete next contribution
Shared API correctness	Scope history and propagate configuration
Patch review	Validate completed comparisons on real series
Required coverage	Define jobs, platforms and completion rules
Review integration	Connect reports to maintainer workflows

Both example plugins provide starting points for this work.

Decisions for this MC

- Which real series or release should we validate next?
- Which jobs and platforms are required for that workflow?
- Which guarantees should become part of the shared Python API?
- Who can review the resulting evidence with us?

[kci-dev](#) / [kci-patchwork](#) / [kci_release_review](#)